Course: CS420 – Artificial Intelligence

Midterm examination review – Lectures 0 to 3

PRACTICE MIDTERM EXAMINATION – TEST 3

Duration: 120 minutes · Closed book · Calculator allowed

Full name: _____ Student ID: _____

Unless a question says otherwise, ties are broken in alphabetical order (left-to-right in trees), and successors are generated in alphabetical order.

Question 1. *Tower of Hanoi with 3 pegs and 3 disks.* Three disks of different sizes are stacked on peg 1 (largest at the bottom). The goal is to move the stack to peg 3, moving one disk at a time and never placing a larger disk on a smaller one.

(a) Consider a state as a tuple of values. Which values should be included in the tuple?

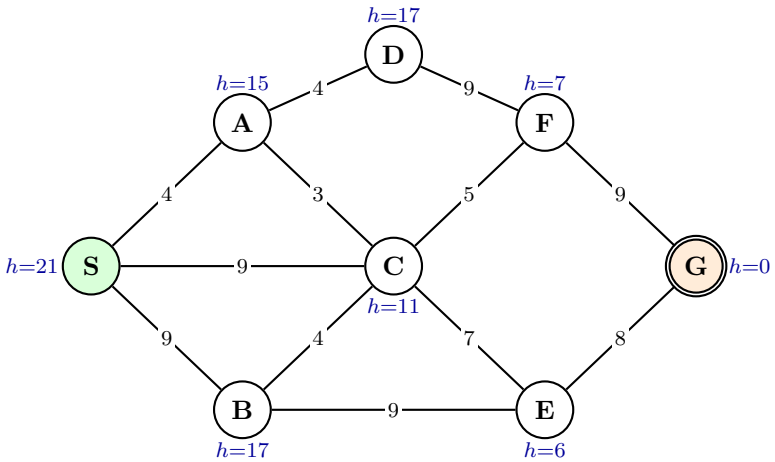
(b) Define the initial state and the goal state.

(c) Define the action function.

(d) What is the action cost?

(e) What is the maximum number of states? How many are reachable? What is the length of an optimal solution?

Question 2. Consider the undirected graph below; S is the initial state and G the goal state.



For each search strategy, (a) list, in order, the states **expanded**, (b) give the path found and (c) give the final content of the frontier (a state is expanded when it is removed from the frontier). Conventions:

BFS, DFS and greedy best-first search test a state for the goal when it is *generated* (so the goal is not expanded); UCS and A* test it when it is *removed* from the frontier (graph search; a cheaper path to a frontier state replaces its entry). DFS avoids loops by remembering the states on the current path only. The path returned will not be accepted if the list of expanded states is wrong.

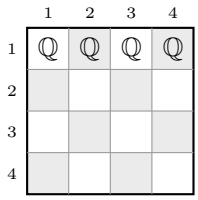
Algorithm	**List of expanded states**	**Path returned**	**Final frontier**
Breadth-first search			
Uniform-cost search			
Depth-first search			*(not asked)*
Greedy best-first search			
Graph-search A*			

(d) Is h admissible? Justify.

(e) Is h consistent? List every violated inequality.

(f) Decrease as few h-values as possible so that h becomes consistent. Does the new heuristic dominate the old one, or the opposite?

Question 3. The 4-queens problem asks to place four queens on a 4×4 board so that no queen attacks another. States are configurations with all four queens on the board, one per column; a successor moves one queen to another square of its column; h(state) is the number of attacking pairs. The initial state S (written as rows $1, 1, 1, 1$; row 1 at the top) is shown below.

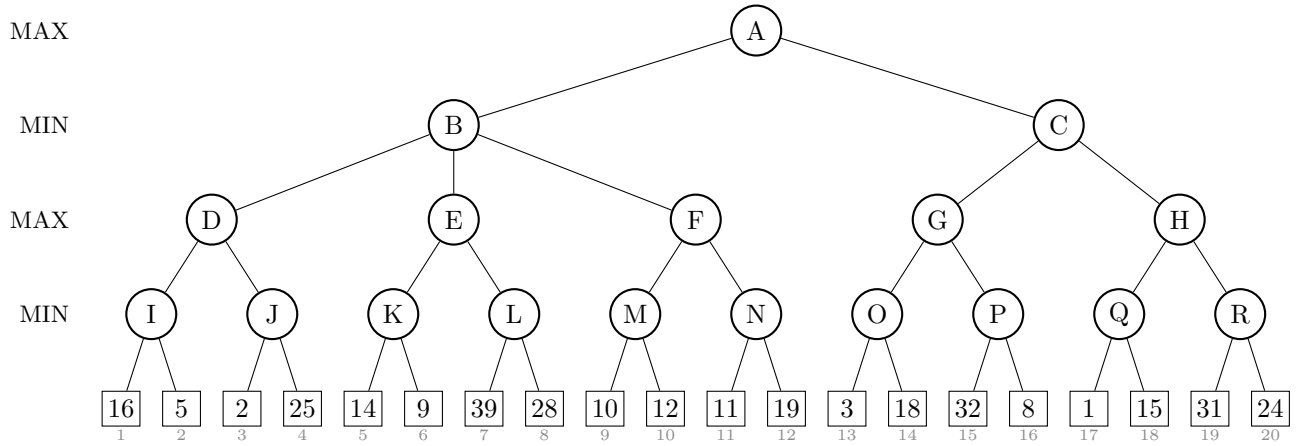


(a) How many states are there in total? How many successors does each state have? Explain.

(b) What is $h(S)$? Explain.

(c) Apply steepest-descent hill climbing from S. At every step write the h value of each of the 12 successors on the board (in the square the queen would move to) and choose the best one (ties: left-most column, then top row). Stop at a solution or when no successor is better.

Question 4. Consider the following 4-ply game tree (MAX at the root, children visited left to right).



(a) Compute the final backed-up value of every node using the minimax algorithm (no alpha-beta pruning at this step). Which move should MAX make at the root?

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
Minimax									

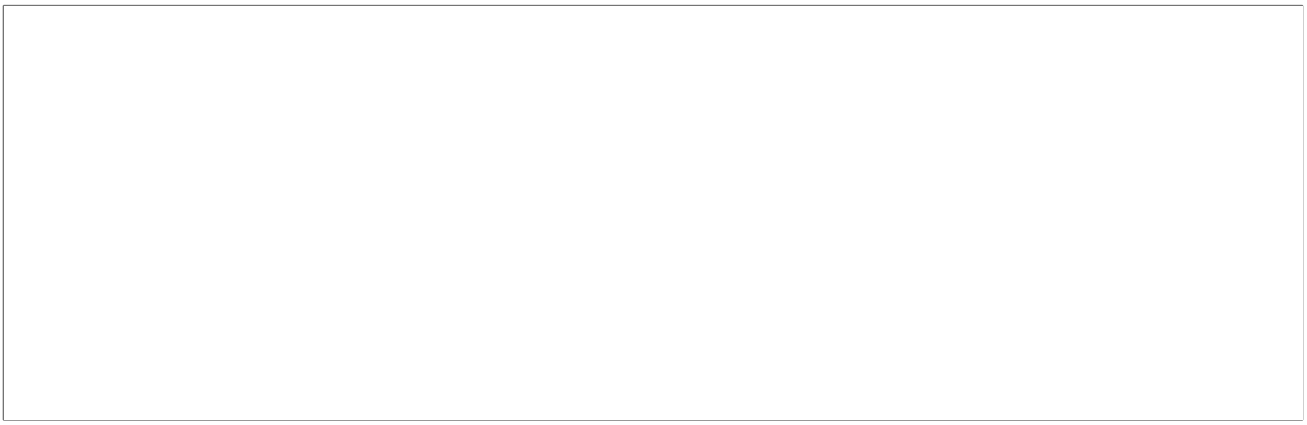
Node	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**
Minimax									

(b) Run alpha-beta pruning (children visited left to right). Give the value computed at every node (mark **X** if the node is never visited), cross out the pruned branches on the tree and briefly explain each case.

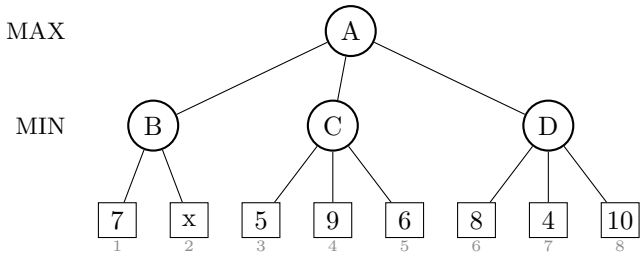
Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
α-β									

Node	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**
α-β									

(c) Using the minimax values from (a), and without performing any alpha-beta calculation, rotate the children of each node at every level so that alpha-beta prunes as much as possible. Draw the new tree (keep the node letters).



Question 5. The game tree below contains an unknown integer leaf value x. Alpha-beta visits children left to right.

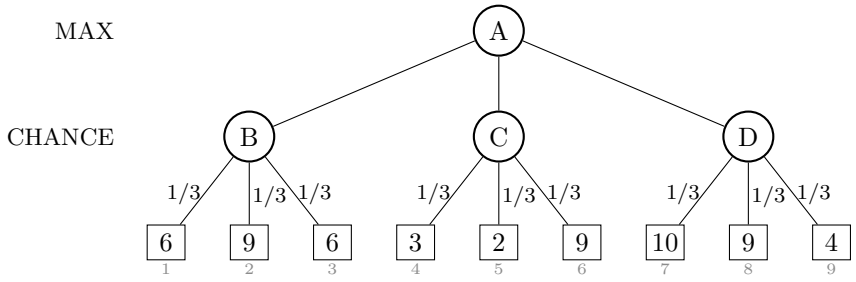


(a) For which values of x are leaves #4 and #5 pruned?

(b) Is leaf #8 ever evaluated?

(c) Give the root value as a function of x and say for which x MAX chooses B.

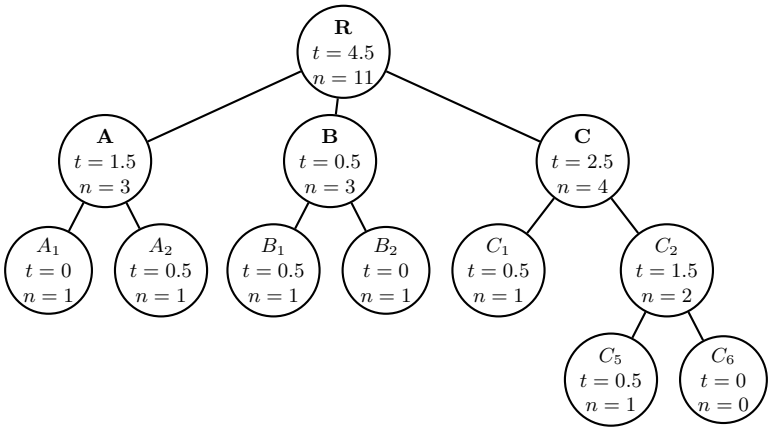
Question 6. Consider the expectimax tree below; each chance node has three equally likely outcomes.



(a) Calculate the expectimax value of every node.

(b) Now assume every leaf value is at most 10 (and at least 0). Evaluating leaves left to right, identify the branches that can be pruned and explain each.

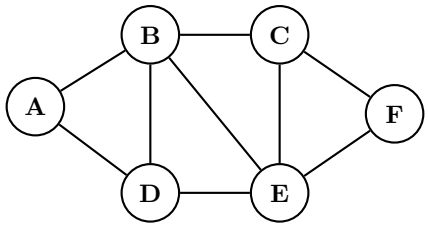
Question 7. Rollouts return 1 (win), 0.5 (draw) or 0 (loss) for the root player. Use UCB1 = $\bar{V}_i + C\sqrt{\ln N/n_i}$ (unvisited = ∞, ties to the left; a never-visited leaf is rolled out directly, otherwise all its children are added and the rollout starts at the first; $X_1 \to X_3, X_4$ and $X_2 \to X_5, X_6$ for $X \in \{A, B, C\}$). The current tree is:



(a) With $C = \sqrt{2}$, calculate UCB1 of A, B, C. Which child is selected? For which values of C would the root select C instead of A?

(b) Perform three iterations whose rollouts return 0, 0.5, 0 (in order); update the statistics and draw the final tree.

Question 8. This problem is about map colouring. Each region must be coloured Red (R), Green (G) or Blue (B), and neighbouring regions must have different colours. The constraint graph is shown below. The following questions are mutually independent.



(a) Cross out all values that would be eliminated by *forward checking* after variable B has been assigned as shown.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	R G B	**R**	R G B	R G B	R G B	R G B

(b) A and B have been assigned as shown, but no constraint propagation has been done yet. **Cross out** all values that would be eliminated by *arc consistency* (AC-3).

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	**G**	R G B	R G B	R G B	R G B

(c) Variable A is already assigned and constraint propagation has been done. **Circle** all unassigned variables that might be selected by the *Minimum-Remaining-Values* (MRV) heuristic.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	R G B	G B	R G B	R G B

(d) Same situation. **Circle** all unassigned variables that might be selected by the *Degree Heuristic*.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	R G B	G B	R G B	R G B

(e) Assume no variable has been assigned yet. Solve the CSP using backtracking with *forward checking*, choosing variables with MRV, then the degree heuristic (DH), and finally alphabetical order; try values in the order R, G, B. For every step give the MRV value of every region not yet coloured, and the DH values of the regions tied on the minimum MRV.

Step 1

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV						
DH						

Assign to the variable _____ with the value _____

Step 2

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV						
DH						

Assign to the variable _____ with the value _____

Step 3

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV						
DH						

Assign to the variable _____ with the value _____

Step 4

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV						
DH						

Assign to the variable _____ with the value _____

Step 5

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV						
DH						

Assign to the variable _____ with the value _____

Step 6

Assign to the variable _____ with the value _____

Question 9. Cryptarithmetic:
$$\begin{array}{r} T\,O \\ +\,G\,O \\ \hline O\,U\,T \end{array}$$
— distinct letters are distinct digits and leading digits are non-zero.

(a) Formulate the CSP: variables (including carries), domains and constraints.

(b) Solve it by constraint propagation, explaining each deduction.

Question 10. *Lab scheduling.* Five lab sessions must each be run by one TA, and a TA cannot run two overlapping sessions: P1 8:00–9:30, P2 9:00–10:30, P3 9:30–11:00, P4 10:30–12:00, P5 11:30–13:00 (sessions that only touch do not overlap). TA **H** can run P1, P2, P4, P5; **L** can run P2, P3; **M** can run P1, P3, P4.

(a) Formulate the CSP (one variable per session; apply the availability constraints to the domains) and draw the constraint graph.

Variables	**P1**	**P2**	**P3**	**P4**	**P5**
Domains					

(b) Run AC-3 with the initial queue $P1 \to P2$, $P2 \to P1$, $P2 \to P3$, $P3 \to P2$, $P3 \to P4$, $P4 \to P3$, $P4 \to P5$, $P5 \to P4$. Cross out the values it eliminates from your domains in (a), and give the solution.

Question 11. Answer the following questions briefly.

(a) Match each field with its contribution to AI: (1) Philosophy, (2) Mathematics, (3) Economics, (4) Neuroscience, (5) Psychology, (6) Control theory. (a) neurons as information-processing units; (b) utility and decision theory; (c) logic, reasoning, mind as a physical system; (d) maximising an objective function over time; (e) formal proof, algorithms, (in)tractability, probability; (f) how people perceive, process information and represent knowledge.

(b) Order chronologically and explain the AI winter: Deep Blue beats Kasparov; Dartmouth meeting; McCulloch & Pitts; AlphaGo beats Lee Sedol; Turing's "Computing Machinery and Intelligence"; the expert-systems boom.

(c) For the 8-puzzle below compute h_1 (misplaced tiles) and h_2 (total Manhattan distance). Which heuristic dominates the other, and what does dominance imply for A*?

Start:

4	1	2
	8	7
6	3	5

Goal:

	1	2
3	4	5
6	7	8

(d) A CSP has $n = 5$ variables with domain size $d = 3$. How many leaves does a naive search tree (any unassigned variable at each level) have, and how many with backtracking? Which property justifies the difference?

Question 12. True or false? For each statement, circle **T** or **F** and justify your answer in one or two sentences (an answer without a correct justification earns no credit).

1. Uniform-cost search expands nodes in order of their optimal path cost. **T** **F**

2. A* is optimally efficient among optimal algorithms using the same heuristic. **T** **F**

3. If $h(n) = 0$ for every node, A* behaves exactly like UCS. **T** **F**

4. If h_2 dominates h_1 and both are consistent, A* with h_2 never expands more nodes than A* with h_1 (ignoring ties). **T** **F**

5. Local beam search with k states is the same as k independent random-restart hill climbs run in parallel. **T** **F**

6. The amount of alpha-beta pruning does not depend on the move ordering. **T** **F**

7. Against a sub-optimal MIN player, the minimax move always gives MAX the best possible outcome. **T** **F**

8. MCTS grows its tree symmetrically, level by level. **T** **F**

9. Node consistency concerns binary constraints. **T** **F**

10. The degree heuristic is mainly used as a tie-breaker for MRV. **T** **F**